

TD1 — Architecture d'un ordinateur : Little Man Computer

M1105 – Système d'exploitation

Éric Würbel

Introduction : Little Man Computer

La séance d'aujourd'hui aura pour but de comprendre le fonctionnement d'un ordinateur fictif à vocation pédagogique : Little Man Computer (LMC). L'objectif est de vous faire comprendre la façon dont fonctionne un ordinateur utilisant l'architecture de Von Neumann (c'est à dire l'architecture de tous les ordinateurs actuels).

Le but n'est pas de faire de vous des spécialistes de la couche matérielle, mais bien de comprendre suffisamment le fonctionnement de ce matériel de façon à prendre conscience de ce qu'un système d'exploitation doit prendre en charge.

Architecture générale

LMC reprend les éléments d'un ordinateur tels que nous les avons vu en cours, en simplifiant les choses. Son architecture est décrite dans la figure 1.

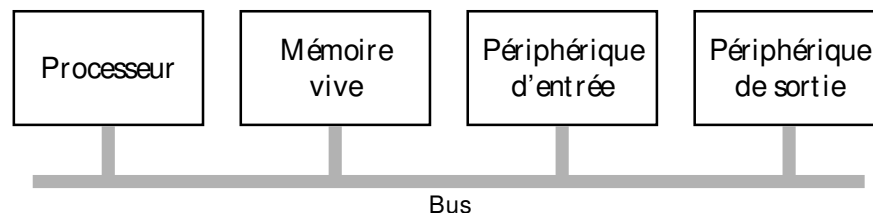


Figure 1 – Little Man Computer : un modèle simplifié d'ordinateur avec architecture de Von Neumann

La mémoire

La mémoire de LMC est très simple : elle est constituée de 100 cases consécutives, numérotées de 0 à 99. Chaque case peut contenir un nombre entier de 3 chiffres, (de 000 à 999 donc).

La mémoire contiendra aussi bien des données que des programmes. Il faudra donc faire attention à ne pas tout mélanger.

Le processeur

Le processeur est constitué de :

- un registre appelé accumulateur (Acc), pouvant contenir un nombre entier à trois chiffres;
- Deux registres pour la manipulation des instructions :
 - un registre appelé compteur ordinal (PC : program counter), qu'il est possible de remettre à zéro. Ce registre contient l'adresse de la prochaine instruction que va exécuter LMC. Ce registre est normalement incrémenté de 1 après chaque instruction exécutée. Ainsi, l'exécution des instructions est séquentielle. Nous verrons qu'il existe des instructions permettant de forcer la valeur du compteur ordinal, et ainsi d'indiquer où continuer l'exécution, appelées instructions de branchement. Ce branchement peut être forcé (branchement inconditionnel) ou soumis à une condition (branchement conditionnel) ;
 - un registre d'instruction (IR : instruction register) contenant une instruction à exécuter
- un registre d'état (nous y reviendrons plus tard) ;
- une Unité Arithmétique et Logique (UAL) capable d'effectuer des additions et des soustractions.
- Une Unité de Commande chargée de coordonner les opérations dans le processeur comme nous l'avons vu en cours. Dans les fait, l'unité de commande, ce sera nous!

L'exécution d'un programme commence toujours à l'adresse 0. Le cycle d'exécution effectué par l'unité de commande est toujours le même, et consiste à répéter indéfiniment (ou jusqu'à l'arrêt de LMC) :

1. Lire le registre PC, aller à l'adresse mémoire correspondante, récupérer l'instruction et la mettre dans IR (Fetch).
2. Incrémenter le registre PC.
3. décoder l'instruction
4. exécuter l'instruction

Les instructions du processeur LMC

Une instruction est un nombre compris entre 000 et 999. Autrement dit, une instruction peut être stockée dans un emplacement mémoire.

Chaque instruction est composée de deux parties :

Table 1 – Little Man Computer : jeu d'instructions du processeur

Code Numérique	Code Mnémonique	Instruction	Description
1xx	ADD	ADD	Ajoute la valeur présente à l'adresse xx de la mémoire à la valeur actuellement dans l'accumulateur. Le résultat se trouvera dans l'accumulateur. Si la somme est supérieure à 999, la valeur de l'accumulateur n'est pas modifiée.
2xx	SUB	SUBTRACT	Soustrait la valeur présente à l'adresse xx de la mémoire à la valeur actuellement présente dans l'accumulateur. Le résultat se trouvera dans l'accumulateur. Si le résultat est négatif, la valeur de l'accumulateur n'est pas modifiée, et un drapeau NEG est armé dans le registre d'état.
3xx	STA	STORE	Enregistre le contenu de l'accumulateur à l'adresse xx de la mémoire (opération destructive). Le contenu de l'accumulateur n'est pas modifié.
5xx	LDA	LOAD	Charge la valeur présente à l'adresse xx dans l'accumulateur. La valeur à l'adresse xx n'est pas modifiée, mais le contenu de l'accumulateur est écrasé.
6xx	BRA	BRANCH	Branchement inconditionnel. Fixe le compteur ordinal PC à l'adresse xx donnée. L'exécution de la prochaine instruction se fera donc à cet emplacement.
7xx	BRZ	BRANCH IF ZERO	Branchement conditionnel. Si l'accumulateur contient la valeur 000, fixe le compteur ordinal PC à la valeur xx. Ne fait rien autrement.
8xx	BRP	BRANCH IF POSITIVE	Branchement conditionnel. Si le drapeau NEG n'est pas armé dans le registre d'état, fixe le compteur ordinal PC à la valeur xx. Autrement, désarme le drapeau NEG et ne fait rien d'autre.
991	INP	INPUT	Lit le port d'entrée, réajuste la valeur et la

Adresse	Contenu	Opération	Commentaires
00	504	MÉMOIRE[4] → Acc	Charge la valeur située à l'adresse 4 dans l'accumulateur
01	105	Acc = Acc + MÉMOIRE[5]	Ajoute le nombre situé à l'adresse 5 à la valeur de l'accumulateur. Ranger le résultat dans l'accumulateur
02	306	Acc → MÉMOIRE[6]	Range la valeur actuelle de Acc à l'adresse 6
03	000		Arrête le fonctionnement
04	030		Première valeur à additionner
05	100		Deuxième valeur à additionner
06	000		Emplacement de stockage du résultat. Sa valeur avant exécution importe peu

L'état du système avant l'exécution du programme peut se représenter de la façon suivante :

Mémoire		
00	504	000 PC
01	105	
02	306	000 IR
03	000	
04	030	000 Acc
05	100	
06	000	

On lance maintenant l'exécution du programme. Le registre PC contient l'adresse de la prochaine instruction à exécuter. On charge donc cette instruction dans IR :

Mémoire		
00	504	000 PC
01	105	
02	306	504 IR
03	000	
04	030	000 Acc
05	100	
06	000	

On incrémente de 1 le compteur ordinal PC :

L'instruction est ensuite décodée : Le chiffre le plus à gauche (5) indique qu'il s'agit d'une instruction servant à charger le registre accumulateur Acc avec la valeur contenue à une certaine adresse mémoire. L'adresse est donnée par les deux derniers chiffres (ici 04).

Une fois décodée, l'instruction est exécutée :

Mémoire			
00	504	001	PC
01	105		
02	306	504	IR
03	000		
04	030	030	Acc
05	100		
06	000		

Le registre PC contient l'adresse de la prochaine instruction à exécuter. On charge donc cette instruction dans IR :

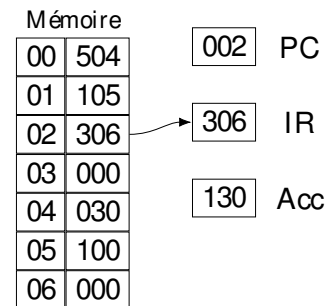
Mémoire			
00	504	001	PC
01	105		
02	306	105	IR
03	000		
04	030	030	Acc
05	100		
06	000		

On incrémente de 1 le compteur ordinal PC :

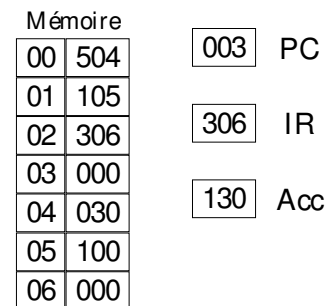
Mémoire			
00	504	002	PC
01	105		
02	306	105	IR
03	000		
04	030	30	Acc
05	100		
06	000		

L'instruction est ensuite décodée : Le chiffre le plus à gauche (1) indique qu'il s'agit d'une instruction servant à additionner au contenu du registre accumulateur Acc la valeur contenue à une certaine adresse mémoire. L'adresse est donnée par les deux derniers chiffres (ici 05).

Le registre PC contient l'adresse de la prochaine instruction à exécuter. On charge donc cette instruction dans IR :

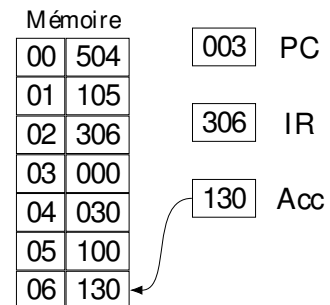


On incrémente de 1 le compteur ordinal PC :



L'instruction est ensuite décodée : Le chiffre le plus à gauche (3) indique qu'il s'agit d'une instruction servant à stocker le contenu du registre accumulateur Acc à une certaine adresse mémoire. L'adresse est donnée par les deux derniers chiffres (ici 06).

Une fois décodée, l'instruction est exécutée :



Le registre PC contient l'adresse de la prochaine instruction à exécuter. On charge donc cette

On incrémente de 1 le compteur ordinal PC :

Mémoire			
00	504	004	PC
01	105		
02	306	000	IR
03	000		
04	030	130	Acc
05	100		
06	000		

Le décodage de l'instruction 000 indique que la machine doit s'arrêter.

Le programme termine donc son exécution, et le résultat du programme se trouve à l'adresse 06 de la mémoire.

Exercice 1

Écrire un programme pour la machine LMC qui :

- lit deux nombres sur le périphérique d'entrée,
- soustrait le second nombre au premier,
- envoie le résultat sur le port de sortie.

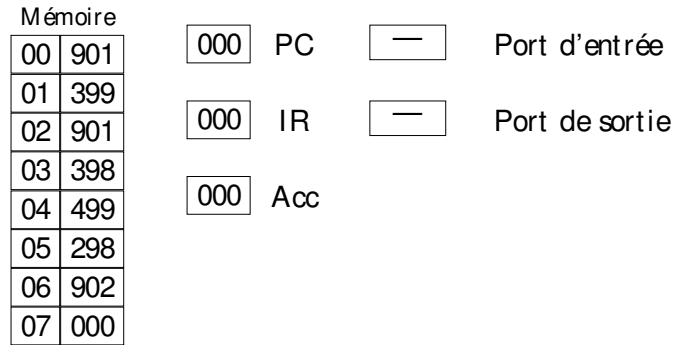
Correction de l'exercice 1

Voici une solution possible pour cet exercice :

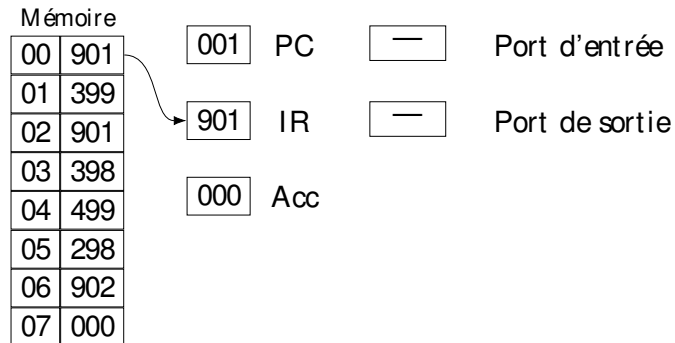
adresse	contenu	mnémonique	commentaire
00	901	INP	Lecture d'un nombre sur le port d'entrée
01	399	STA 99	Écrit le nombre lu à l'adresse 99
02	901	INP	Lecture d'un nombre sur le port d'entrée
03	398	STA 98	Écrit le nombre lu à l'adresse 98
04	599	LDA 99	Charge la valeur située à l'adresse 99 dans Acc
05	298	SUB 98	Acc - valeur à l'adresse 98
06	902	OUT	Affichage
07	000	HLT	Arrêt

Quelques remarques sur ce programme :

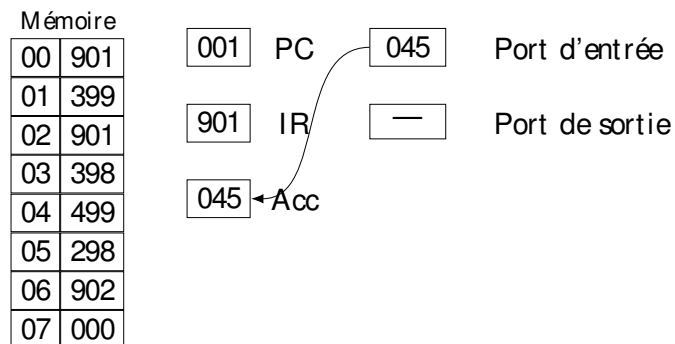
- Remarquez la colonne contenant la notation mnémonique. Pour nous autres humains c'est plus simple à lire, mais évidemment le processeur lui ne comprend que les nombres de la colonne



On lance maintenant l'exécution du programme. Le registre PC contient l'adresse de la prochaine instruction à exécuter. On charge donc cette instruction dans IR, et on incrémente PC :

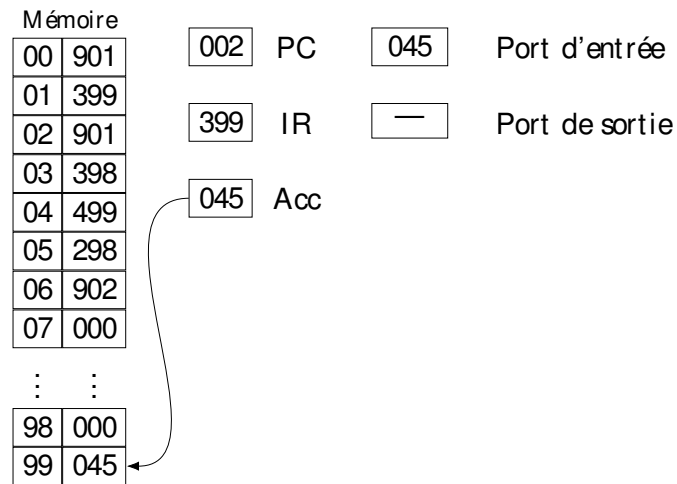


On décode l'instruction : l'instruction 901 (`INP`) lit un nombre sur le port d'entrée (le périphérique d'entrée). Supposons que le nombre 45 soit disponible sur ce périphérique (un utilisateur a tapé 45 au clavier). Cette valeur va être placée dans Acc. Après exécution on a :

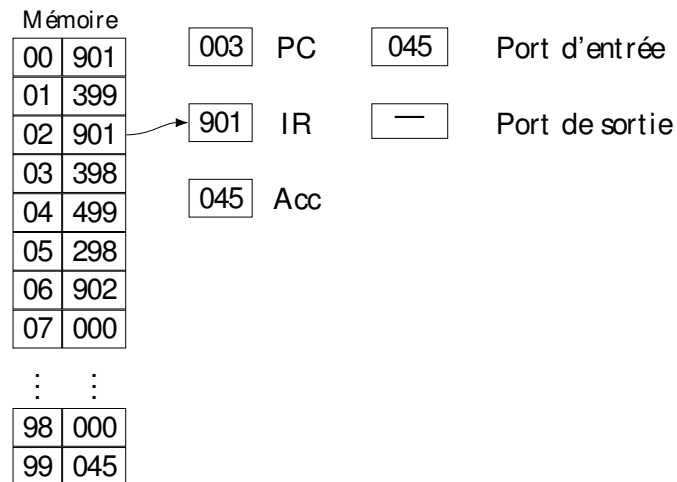


On continue : chargement dans IR de l'instruction située à l'adresse désignée par PC, puis incrémentation de PC :

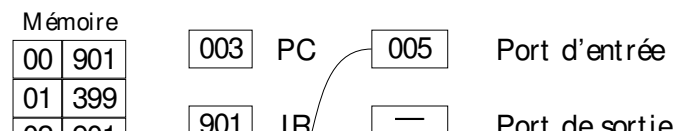
Décodage de l'instruction dans IR : il s'agit d'une instruction de stockage (3xx, STA) de la valeur de Acc à une adresse mémoire, ici 99. On exécute :



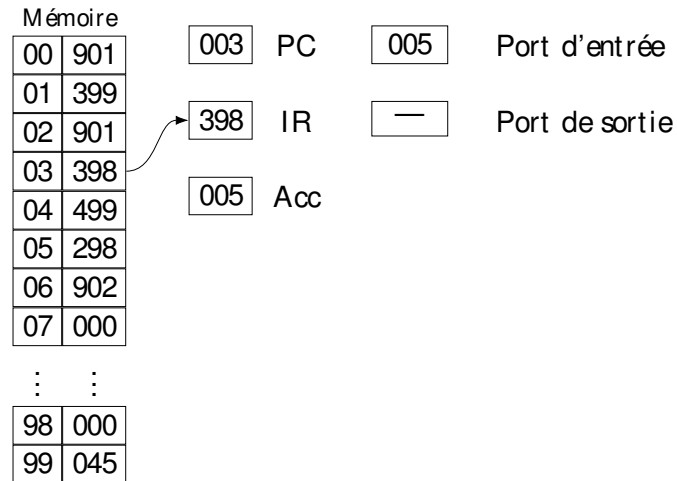
On charge l'instruction située à l'adresse indiquée par PC, puis on incrémente PC :



On décode l'instruction : l'instruction 901 (INP) lit un nombre sur le port d'entrée (le périphérique d'entrée). Supposons que le nombre 5 soit disponible sur ce périphérique (un utilisateur a tapé 5 au clavier). Cette valeur va être placée dans Acc. Après exécution on a :



On charge l'instruction située à l'adresse indiquée par PC, puis on incrémente PC :



Exercice 2

Écrire un programme pour la machine LMC qui :

- lit deux nombres sur le périphérique d'entrée,
- envoie la valeur 1 sur le port de sortie si le premier nombre est supérieur ou égal au second. Dans le cas contraire il renverra zéro.

Remarque

Observez la façon dont vous êtes amené à gérer la mémoire : une partie pour le programme, une partie pour les données (on ne doit pas mélanger les deux). C'est une des tâches fondamentales du système d'exploitation.

Exercice 3

Pour illustrer la remarque précédente, analysez l'exécution du programme suivant. Que se passe-t-il ?

Notez au passage l'utilisation de la notation mnémonique pour ce programme. Pour nous autres humains c'est plus simple à lire, mais évidemment le processeur lui ne comprend que les nombres de la colonne « Contenu ». Cette notation symbolique s'appelle langage d'assemblage, ou assembleur.

Dans cette colonne en assembleur, on voit apparaître une instruction nouvelle, qui n'en est pas une : `DATA`. Ce n'est pas une instruction du processeur, mais une directive d'assemblage permettant

